Talk track

Demo Computer as the service desk teammate

Talk track for DevRev sales engineers.

Use this script when you train another SE or run the **identity recovery** demo for a customer. The environment steps live in se-configuration-guide.pdf (PDF). A print PDF of this talk track is se-demo-script.pdf. Do not walk a customer through Docker, ngrok, or client secrets.

Computer is the AI teammate that remembers the business and **acts** inside guardrails. This meeting proves that sentence for IT: an employee asks for help in plain language, Computer calls a governed skill, and the employee gets back to work.

Audience and timebox

Field	Value
Audience	Sales engineers, solutions consultants, and the customers they enable
Goal	Leave the room able to retell the story and run the demo themselves
Live time	12 to 15 minutes
Questions	5 minutes after the close

Keep the customer version to **12 minutes**. Use the extra time only when you are training another SE.

The story

An employee is locked out of SSO. They do not open a ticket and they do not wait on a queue. They ask Computer. Computer finds the Keycloak account, reports lockout, and unlocks it. If the password also needs to change, Computer sends a Keycloak reset email. Credentials never enter the chat.

That is service desk automation: Computer handles the reset so IT can spend time on work that needs a human.

Name the employee **Daniel**. His DevRev login is `daniel.carvajal@devrev.ai`. Keycloak stores him as username `danielcarvajal` with email `daniel.carvajal@devrev.com`. If someone asks why those differ, use Identity mapping. Do not open with that detail.

Brand voice in the room

Write and speak the way DevRev writes: knowledgeable, approachable, and forward-thinking. Use American English. Use sentence case on slides and whiteboards. Prefer active voice.

Say **Computer**, **DevRev**, and **PLuG** exactly that way. Computer is a teammate, not a widget. PLuG is the chat surface, not "the chatbot."

Lead with the outcome. Then show the control: skills, permissions, and a log you can inspect. The product line is "Computer acts. You stay in control."

Say	Do not say
Computer	The chatbot, Devrev AI, ChatGPT for IT
Computer skill	Random tool the model invented
Service desk automation	A cool hack we wired up
Guardrails	Trust us, it is safe
DevRev	Devrev, DEVREV, devrev
PLuG	Plug, PLUG
Employee gets back to work	We scraped an admin API

Do not use slang. Do not title slides in all caps. Do not read a client secret, personal access token, or raw JWT on the call.

How the two documents work together

1. The night before, follow se-configuration-guide.pdf through the pre-meeting checklist.
2. In the meeting, use **this** script only.
3. After the meeting, send the configuration guide to any SE who will rebuild the environment.

Pre-call checklist

Complete every item in the configuration guide's checklist before a customer meeting. In the last ten minutes before you join:

1. Confirm Docker Keycloak and ngrok are still up.
2. Open a **new** Computer chat in dcm-test. An old session can keep a stale skill schema.
3. Type `check danielcarvajal` once, off camera. You want enabled and lockout status, not a request for a URL or secret.
4. Decide the password-reset path: - **Email path:** realm SMTP works. You will ask Computer to reset the password. - **Internal path:** SMTP is not configured. You will show `/reset_password`

`danielcarvajal --temp` on a ticket and say the temporary password is an **internal** comment.

5. Keep Keycloak Admin and the ngrok inspector on a second screen. Do not share that screen unless a technical buyer asks to see the calls.
6. Close any tab that shows a client secret, PAT, or JWT.

If the ngrok host changed overnight, stop. Update the snap-in connection and the three Computer skills, then re-check. Do not join with a stale host.

Cast and stage

You play two roles. Say which one you are in before each scene.

Role	Who	What they do
Employee	Daniel	Asks Computer for help in plain language
Sales engineer	You	Frames the story, then shows the same action on a ticket if asked

Primary surface: Computer in the DevRev web app (search bar, slug `computer`). Optional surface: a ticket discussion with `/check_account`, `/unlock_account`, and `/reset_password`.

Leave the Keycloak admin console off the shared screen unless you need proof that the account changed.

Scene-by-scene

Times are a guide. If Computer is slow, narrate what the skill is doing instead of filling silence with implementation detail.

Scene 1. Frame the problem (90 seconds)

Say

> Most IT tickets do not need a human. They are resets, lockouts, and access requests. Computer is the AI teammate that takes those actions inside the guardrails your org sets. We are going to lock an employee out of SSO, ask Computer for help in plain language, and watch Computer recover the account. You will not see a password in this chat.

Show

The DevRev home for org **dcm-test**. Do not start in Settings.

Do not say

“We stood this up on my laptop with Docker and a tunnel.” Save that for an SE lab.

Scene 2. The employee is stuck (2 minutes)

Do this off to the side, or skip if you already locked `testuser`.

In Keycloak, sign in as `testuser` / `testuser@yourcompany.com` and fail the password three times so brute-force lockout is on. For Daniel, you can skip the failed logins and still **check** the account. Unlock is more dramatic when lockout is true.

Say

> Daniel cannot get into work. In a typical help desk, this is a ticket, a queue, and a wait. Here, Daniel already has Computer.

Click

Open Computer. Start a **new** chat.

Scene 3. Check the account (3 minutes)

Type, as Daniel

```
I'm locked out. Can you check my Keycloak account?
```

If Computer asks who you are, follow with:

```
check danielcarvajal
```

Show

Computer reports whether the account is enabled and whether brute-force lockout is on. Point at the result, not at the HTTP path.

Say

> Computer did not guess. It used a Computer skill your admin published: KeycloakCheckAccount. The model only passed an identity. The method, URL, and service credentials stay on the skill. That is a guardrail, not a prompt trick.

If Computer asks for a method, URL, or secret, **stop the scene**. The skill is stale or the chat is old. Start a new chat. If that fails, move to the ticket command in Scene 6 and book a lab follow-up.

Scene 4. Unlock (3 minutes)

Type

```
unlock my account
```

or, if Computer needs a name:

```
unlock danielcarvajal
```

Show

Computer confirms the lockout is cleared. If you locked `testuser` instead, use `unlock testuser` and say you are helping a named employee.

Say

> This is the difference between an assistant that writes a reply and a teammate that acts. Computer called KeycloakUnlockAccount, cleared the lockout counter, and re-enabled the user. Permanent lockout here is `enabled: false` after three failed logins — unlock lifts that. Consequential actions stay on skills your org tested and published.

If a technical buyer asks "prove it"

Switch to Keycloak Admin → Users → `danielcarvajal` (or `testuser`) and show the account is enabled. Switch back immediately. Do not linger in the admin console.

Scene 5. Reset the password or use the backup (2 minutes)

Email path (SMTP is configured)

Type

```
reset my password
```

Say

> Computer sent Keycloak's UPDATE_PASSWORD email. The employee finishes the reset in Keycloak. The password never appears in Computer. That is the path you want in production.

Internal path (no realm SMTP)

Do not apologize at length. Move to a ticket discussion and type:

```
/reset_password danielcarvajal --temp
```

Say

> This realm has no SMTP, so the snap-in set a temporary password and posted it as an **internal** comment. In a customer meeting you would never read that password on a public thread. Production uses the email action. The temporary path is for labs and break-glass.

Never paste the temporary password into Computer chat or a customer- visible comment.

Scene 6. Optional: the same work on a ticket (2 minutes)

Use this scene when the buyer lives in a ticket queue, or when you are training an SE who will support both surfaces.

Click

Open any ticket in **dcm-test**. Open **Discussions**.

Type

```
/check_account danielcarvajal
/unlock_account danielcarvajal
```

Say

> Same recovery path, different door. Computer is the employee experience. The Keycloak Password Reset snap-in is the agent experience on the work item. Both call the same identity store. Your service desk does not maintain two sources of truth.

Skip this scene if you are already at 12 minutes and the Computer path worked.

Scene 7. Close (90 seconds)

Say

> That is the story. An employee asked for help. Computer understood the request, used a published skill, and took action inside guardrails. IT did not get another password ticket. Your team keeps control: which skills ship, who they may act for, and what is logged.
>
> If you want to rebuild this environment, we have a step-by-step configuration guide for sales engineers. I will not walk credentials in this room.

Stop. Take questions. Do not add a sixth tool or a second identity provider unless someone asks.

Identity mapping

Use this only when someone notices the mailbox mismatch, or when you train an SE who will operate **dcm-test**.

Say

> DevRev login and Keycloak are not the same mailbox in this lab. Daniel signs in to DevRev as `daniel.carvajal@devrev.ai`. Keycloak stores `daniel.carvajal@devrev.com` and username `danielcarvajal`. Computer tries the DevRev email, then the `@devrev.com` alias, then the username, including a hyphen-stripped form of the display name. A search for `@devrev.ai` alone returns no user. That is expected.

What Daniel says	What you type if Computer needs help	What Keycloak has
Unlock my account	`unlock danielcarvajal`	Username `danielcarvajal`
Check my email	`check daniel.carvajal@devrev.com`	Email on `@devrev.com`
Unlock Daniel-Carvajal	`unlock daniel-carvajal`	Same user; hyphens stripped

A requester may manage **their own** account. Do not reset a different person unless the requester clearly names that person's email or username.

If a scene fails

Stay in the story. Name the backup. Do not debug JSONata on the call.

What you see	What you do	What you say
Computer asks for a URL or secret	New chat, then `check danielcarvajal`	"The skill keeps credentials off the model. I am opening a fresh session so Computer picks up the published skill."
"No Keycloak user" for Daniel	Retry with `danielcarvajal`	"Computer maps the DevRev login to the identity store. I am passing the username explicitly."
Computer cannot reach Keycloak	Ticket command, or reschedule	"The identity store is on a lab tunnel for this demo. In production this is your IdP URL."
Reset email fails	`/reset_password … --temp` on a ticket	"This lab realm has no SMTP. Production sends Keycloak's reset email. The temporary password stays internal."
Unlock does nothing visible	Check `testuser` after three failed logins	"I will lock a lab user so you can see brute-force status change."

If two scenes fail, close on the architecture slide: Computer skill, snap-in command, Keycloak Admin API. Offer a working lab the next day.

Objection handles

Answer in two sentences. Offer the configuration guide for depth.

"Is this a chatbot that wraps Keycloak?"

Computer is a teammate with skills. The model chooses **when** to check or unlock. The skill defines **how**. Your org publishes that skill before anyone uses it.

"Where do the credentials live?"

On the snap-in connection and the skill workflow. Computer receives an email or username. It does not receive the client secret.

"What stops Computer from unlocking the wrong person?"

The requester may recover their own account. Recovering someone else requires a named email or username. Every run is logged. You can require approval on the consequential skill if that is your policy.

"We use Okta / Microsoft Entra, not Keycloak."

Treat Keycloak as the stand-in identity store. The story is the same: Computer skill, governed credentials, employee language. Swap the HTTP steps for the IdP you run.

“How long does this take to stand up?”

This lab is a laptop Keycloak, a tunnel, and three published skills. A production path uses your IdP URL, your SMTP, and Agent Studio so IT owns the skill. The configuration guide lists the lab steps.

“Can we see the API?”

Yes, after the story. Show the ngrok inspector: token, search, lockout. Do not show the Bearer token value.

After the meeting

Send SEs these two links only:

1. se-demo-script.pdf — this talk track
2. se-configuration-guide.pdf — how to rebuild the lab

Point implementers at keycloak-password-reset/README.md for snap-in commands and local Keycloak. Do not send the org PAT, the Keycloak client secret, or a captured JWT.

Practice loop for SE training

Run this once with another SE before you take it to a customer.

1. Trainer plays the buyer. Trainee plays the SE.
2. Trainee delivers Scenes 1 through 5 without notes for more than a glance.
3. Trainer asks two objections from the list above.
4. Trainer fails SMTP on purpose. Trainee must hit the `--temp` backup without breaking character.
5. Debrief on language: count “chatbot,” “hack,” and any secret that appeared on screen. The target is zero.
